

The Essence of Event-B from an Abstract Interpretation Perspective

Patrick Cousot [0000-0003-0101-9953]

Courant Institute, NYU, New York, NY, USA, pcousot@cims.nyu.edu

Abstract. We explain the basic concepts of Event-B, that is the specification of machines and refinement, in light of abstract interpretation. After formally defining the trace semantics of systems handled by Event-B, we show that machines are specifications of abstractions of this semantics while refinement is an inverse abstract interpretation reducing the degree of nondeterminism of this trace semantics. We conclude by proposing in induction principle for program verification exploiting both property abstraction and program refinement.

Keywords: Event-B · Abstraction · Abstract machine · Semantics · Refinement · Verification · Abstract interpretation · Formal methods.

1 Introduction

I have had the pleasure to have my office close to that of Jean-Raymond ABRIAL during my thesis in Grenoble and to share a common passion for climbing, mainly in the Vercors limestone massif and the French and Swiss granite and glacier Alps. At the time Jean-Raymond was working on *data semantics* [1] and started to be increasingly interested in formal program verification and construction. He had strong ideas on computer science, in particular program design and verification, that he explaining very clearly, and was always ready for discussion on many subjects with a young ignorant trying to understand abstraction.

When Jean-Raymond left Grenoble, I was given the charge of his class on data structures, programming, and verification that gave me a strong taste for teaching, in particular formal methods.

We met later in conferences, meetings, summer schools, and seminars in particular during his stay at ETH Zurich and sometimes also by chance, like in Venice, on the Ponte dell’Accademia, where, for a long time on the bridge, we passionately discussed the genius of Venice architecture and urbanism design.

Unfortunately, Jean-Raymond left Grenoble before I had established the foundations of abstract interpretation with Radhia and therefore we have never had scientific collaborations. So I was surprised to be invited by Egon Boerger and Yamine Aït-Ameur to contribute to this Gedenkschrift with the proposed theme of “Relating program verification by abstract interpretation with proofs in Event-B”. Since no one, including myself, seemed to have understood the correspondence during the “Workshop on Refinement and Abstraction” (ETL

Osaka, Japan, 15–17 November 1999), and I had made no progress since then, I thought taking on the challenge would be interesting.

Although I had read the B-book [2] decades ago, I had to learn Event-B, almost from scratch [4,7] while knowing that I could not absorb the abundant literature on Event-B (event-b.org) in the allotted time.

It turns out that I could slightly extend the challenge to an explanation of the semantics of Event-B in terms of abstract interpretation [17]. Of course this brings nothing new to Event-B but this formalization helps me to better understand two related forms of abstractions when respectively applied to models (as in Event-B) and to properties of these models (as in abstract interpretation). It might also be useful to conceive Event-B refinements by calculational design of the concrete from the given abstract using a formal correspondence between their semantics (such as a Galois connection or equivalent).

Event-B [4] is an evolution of the B method [2] to specify a reactive system and provide “a mathematical model of its dynamic behaviour” [3,7]. The description is reduced to contexts and machines. It proposes to observe states via events and reuses B’s refinement.

The behavior of a reactive system is described by a concrete machine, more precisely its semantics defining the possible executions of machines, a name for a computer system. This semantics, described in part section 2, is a finite or infinite set of traces which is the mathematical model of the system dynamic behavior mentioned by Jean-Raymond. This trace specified by abstraction into a machine, as shown in part section 3. The objective of an Event-B development is to ultimately define this concrete semantics through several refinement steps formalized in sections 4 and 5. Finally, section 6 proposes a general induction principle to combine property abstraction and program refinement.

I assume a minimal knowledge of Event-B [4, Ch. 5. “The Event-B modeling notation and proof obligation rules” and Ch. 14. “Mathematical models for proof obligations”], [29,33] and abstract interpretation [13] (in particular the formalization of property abstraction using closure operators or Galois connections [18] as recalled in [15, Ch. 11]). For Galois connections, I write $\langle C, \sqsubseteq \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \leq \rangle$ for $\langle C, \sqsubseteq \rangle$ and $\langle A, \leq \rangle$ are posets, $\alpha \in C \rightarrow A$ is the abstraction, $\gamma \in A \rightarrow C$ is the concretization, and $\forall x \in C, y \in A. \alpha(x) \leq y \Leftrightarrow x \sqsubseteq \gamma(y)$. For upper closure operators (increasing, extensive, and idempotent), $\langle A, \leq \rangle = \langle C, \sqsubseteq \rangle$ and γ is the identity on $C = A$. I use transformers $\text{post}[r] \triangleq \lambda P. \{s' \mid \exists s \in P. \langle s, s' \rangle \in r\}$, $\text{pre}[r] \triangleq \text{post}[r^{-1}]$, $\widetilde{\text{pre}}[r] \triangleq \neg \circ \text{pre}[r] \circ \neg$, and $\widetilde{\text{post}}[r] \triangleq \widetilde{\text{pre}}[r^{-1}]$.

2 The Semantics of Event-B Systems

An (abstract) machine “represents the mathematical model of an entire system as seen from a unique global abstract point of view.” [3, p. 177] (where it was originally called “(abstract) system”). The system is closed in that it does not allow transfer of information in or out of the system. Event-B specifies how to define the semantics of an abstract machine. I adopt the inverse approach,

starting by what should be the semantics of an Event-B system and then, in section 3, how to define it, by abstraction into a machine.

2.1 States and Events. An Event-B specification usually starts by defining *contexts* that is a set of states $S \in \wp(S)$ and operations on these states by means of carrier sets, constants, and axioms (from which theorems can be deduced). The definition of a set of events $E \in \wp(E)$ is a component/clause **events** of a machine. Each event $e \in E$ is specified by an event structure [3, fig. 5.10] that describes a potential transition between a pre-state and a post-state. If there exists values of the parameters (in the clause **any**) for which the predicates of the **where** clause hold of the pre-state then actions (in the clause **then**) specify the transition. These actions [3, Sect. 5.1.8] specify the change from a prestate to a poststate which, in general, is nondeterministic.

Event-B uses set-theory and first-order logic [4, Ch. 9] to define the correct set of state definitions S in contexts [4, Sect. 5.1.3] and correct set of event definitions E in machines [4, Sect. 5.1.7]. To avoid inconsistencies, these set definitions must sometimes include proof obligations, in particular when using partially defined objects (such as division by zero). “*a failed proof reveals a bug*” [4, p. 40]. For simplicity, and since this is a classic problem in mathematics when using partially defined objects, I consider that sets, constants, and basic operations in event guards and actions of Event-B are well-defined by construction (i.e. all well-definedness proof obligations [4, Sect. 5.2.12] have been proved). For other proof obligations, I will show how they derive from the definition of the traces and transition semantics of machines.

Event-B offers the possibility of accompanying definitions in contexts with theorems requiring a proof [4, Sect. 5.2.11]. This is a practical way of organizing proofs that I ignore, as well as how they must be proved. I assume that all theorems in an Event-B definition can be proved, which amounts to saying that the Event-B semantics is well-defined.

So I am interested in the semantics of Event-B more than in its syntax and well-definedness, knowing that I neglect an important aspect of Event-B when implemented in tools like the Rodin platform [5,35].

2.2 Execution Traces. Event-B specifications ultimately define traces [4, Sect. 14.3.2]. Traditionally, traces $s_0 \xrightarrow{e_0} s_1 \xrightarrow{e_1} \dots$ are non-empty sequences of states $s_i \in S \in \wp(S)$ separated by events $e_i \in E \in \wp(E)$ meaning that in prestate s_i event e_i yields poststate s_{i+1} . I encode such traces as a sequence of pairs $\langle s_0, e_0 \rangle \langle s_1, e_1 \rangle \dots$. Traces must start with an initial state and init event [4, Sect. 2.4.16]. The finite traces are

$$\pi \in \mathbb{L}^+(S, E) \triangleq \{ \pi \in [0, n] \rightarrow (S \times E) \mid n \in \mathbb{N} \wedge \pi_0 \in \text{init} \} \quad (1)$$

that is a finite sequence of a state $\text{stof}(\pi_i) \triangleq (\pi_i)_1$ in S followed by an event $\text{evtof}(\pi_i) \triangleq (\pi_i)_2$ in E , $i \in \mathbb{N}$ and starting from an initial event (that I replace by a pair of initial state and an initial event in init for uniformity of the trace definition). I call such pairs $\langle s, e \rangle \in S \times E$ trace transitions. The trace transitions $\dots \langle s, e \rangle \langle s', \dots \rangle \dots$ in a trace follow from the definition of the event e which has a

guard that must hold in state s and a deterministic ($:=$) or (possibly unbounded) nondeterministic ($:$ or $:\epsilon$) action assigning to state s to get successor state s' [4, Sect. 5.1.8]. The relation between the pre and post states can also be specified by a before-after-predicate [4, Ch. 2.4.4].

In Event-B, a trace may be required to be deadlock free [4, Sect. 2.4.21] so that it must be infinite [4, Sect. 14.3.3], since ‘The system always “runs” for ever’ [4, p. 422]. Therefore an Event-B trace is

$$\pi \in \mathbb{T}^\infty\langle S, E \rangle \triangleq \{ \pi \in \mathbb{N} \rightarrow (S \times E) \mid \pi_0 \in \text{init} \} \quad (2)$$

(We only consider infinite traces since finite traces can be encoded as an infinite one with the final state and event repeated for ever.)

2.3 Trace Semantics. Given states $S \in \wp(S)$ and events in $E \in \wp(E)$, a trace semantics T is a finite or infinite set of infinite execution traces on S and E . So T belongs to the domain $\mathbb{T}^\infty\langle S, E \rangle$ which is

$$T \in \mathbb{T}^\infty\langle S, E \rangle \triangleq \wp(\mathbb{T}^\infty\langle S, E \rangle) \setminus \{\emptyset\} \quad (3)$$

where $\wp$ is the powerset. This means that T is a property of the execution traces defined in extension. We denote concatenation of traces or sets of traces by juxtaposition, sometimes using $\cdot$ to make concatenation more explicit.

Notice that this trace semantics covers concurrency which is handled by interleaving in Event-B, that is, assuming sequential consistency [4, Ch. 7]. However, our formalization of Event-B by abstraction of the trace semantics could be applied to weak consistency models by e.g. considering instead abstractions of the anarchic semantics of [8].

Other comparable sequential consistent specification languages like TLA+ [30] and Unity [10] have trace semantics. In Unity, there is an implicit weak fairness hypothesis (if a transition is eventually always enabled, then it is taken infinitely often). There is no such implicit fairness hypothesis in TLA+ and Event-B (see [31] for a comparison of Event-B and TLA+ on the example of distributed termination). In TLA+ the trace semantics is specified using temporal logic. Fairness requirements are specified using temporal formulas (such as WF_{vars} in [31, Fig. 1]). Similarly, Event-B makes no weak or strong process fairness hypothesis for parallel or distributed systems. Fairness has to be explicitly specified during the refinement process [7, p. 87].

2.4 Objective of Event-B. Event-B is a formal method aimed at system-level modeling and analysis whose ultimate goal is to define a trace semantics $T \in \mathbb{T}^\infty\langle S, E \rangle$ of the modeled discrete system by successive refinements starting from an informal, often incomplete if not incorrect, specification.

However, not all arbitrary sets of traces $T \in \mathbb{T}^\infty\langle S, E \rangle$ represent computations of Event-B machines. We express additional requirements on the sets T of traces using closure operators on $\mathbb{T}^\infty\langle S, E \rangle$. Since all traces are infinite in Event-B, these upper or lower closures of trace semantics are much simpler than the general case considered in [23, Sect. 2.6].

2.5 History Independence. It is implicit in Even-B that the future of a computation must depend only on its current trace state, not on how that state was reached. This means that if $\pi_1 \langle s, e \rangle \pi_2$ and $\pi'_1 \langle s, e \rangle \pi'_2$ are two infinite traces of the semantics sharing intermediate state s and (nondeterministic) event e then $\pi_1 \langle s, e \rangle \pi'_2$ and $\pi'_1 \langle s, e \rangle \pi_2$ should also be infinite traces of the semantics [23, Sect. 2.6.5]. Let us define the abstraction (π_1 and π'_1 are finite traces, $\langle s, e \rangle$ is a trace transition, and π_2 and π'_2 are infinite traces)

$$\text{Efus}(\mathsf{T}) \triangleq \{ \pi_1 \langle s, e \rangle \pi'_2 \mid \exists \pi_2, \pi'_1. \pi_1 \langle s, e \rangle \pi_2, \pi'_1 \langle s, e \rangle \pi'_2 \in \mathsf{T} \} \quad (4)$$

Efus is increasing and extensive for set inclusion $\subseteq$ but not necessarily idempotent. Therefore, the closure by fusion [23, Sect. 2.6.5] is defined as

$$\text{Ffus}(\mathsf{T}) \triangleq \text{lfp}^\subseteq \lambda X. \mathsf{T} \cup \text{Efus}(X) \quad (5)$$

where $\text{lfp}^\subseteq F$ is the least fixpoint of the $\subseteq$ -increasing function F on a complete lattice for partial order $\subseteq$ [34]. Ffus repeats the fusion Efus of traces infinitely many times. Ffus is an upper closure operator such that $\text{Efus} \circ \text{Ffus} = \text{Ffus}$ and $\text{Ffus} = \bigcup_{n \in \mathbb{N}} \text{Efus}^n$, pointwise [23, lem. 2.6.5 ~1]. Event-B always requires sets of traces T to be history independent, that is $\text{Ffus}(\mathsf{T}) = \mathsf{T}$. This is an abstraction onto $\mathbb{T}^F \langle S, E \rangle \triangleq \{ \mathsf{T} \in \mathbb{T}^\infty \langle S, E \rangle \mid \text{Ffus}(\mathsf{T}) = \mathsf{T} \}$,

$$\langle \mathbb{T}^\infty \langle S, E \rangle, \subseteq \rangle \xleftrightarrow[\text{Ffus}]{\mathbb{1}} \langle \mathbb{T}^F \langle S, E \rangle, \subseteq \rangle \quad (6)$$

where $\mathbb{1}$ is the identity function.

2.6 Further Hypotheses on the Trace Semantics. There are several different ways to use Event-B to produce a sequential, concurrent or distributed algorithm, which may require additional restrictions on traces. For example, physical systems are generally considered to be finite rather than mathematically infinite so that the nondeterminism of computations is necessarily bounded as considered in section 2.7. In particular, determinism considered in Sect. 2.8 is ultimately necessary to get a sequential algorithm [4, Ch. 15]. Event convergence considered in Sect. 2.10 prevents starvation by requiring that a specific subset of events cannot take control of the system forever so that the other events would be postponed for ever [4, p. 64]. These restrictions are abstractions of the trace semantics and can be composed to get the appropriate trace model of the considered closed system.

2.7 Bounded Nondeterminism. I could not find in [4] the restriction that nondeterminism should be bounded. The number of events is finite, but infinite sets of states do not look to be prohibited.

However, the method for proving inevitability, such as event-convergence, uses variant functions in the naturals or in finite sets since the numeric variant proof obligation rule [4, Sect. 5.2.7] requires the numeric variant function to be a natural number.

A minor point is the variant function must in general depend on the current and initial states [20, p. 290]. This can always be done indirectly by adding auxiliary variables.

A more important point is that the requirement that the numeric variant function be a natural number yields a sound but incomplete proof method for unbounded nondeterminism [23, Th. 5.2.6.3 ~1], for which transfinite ordinals may be required [23, Sect. 5.2.6], [25].

This restriction to naturals may result from the initial decision [7] to reuse the B prover [6]. Moreover, unbounded nondeterminism as well as fairness in parallel or distributed programs can be understood as the limit of bounded nondeterminism and fairness when the unknown bound tends to infinity. This simplifies reasonings by eliminating this unknown bound so that the details of an implementation can be deliberately ignored, at least in the first steps of the Event-B development.

Yet the physical realization of unbounded nondeterminism and fairness is questionable so that a bound is not that unreasonable in practice. Finally, in a development by successive refinements where the final result must be deterministic, the nondeterminism must be bounded at some point in the development, at which point all intermediate sets of traces T in an Event-B development are subject to locally bounded nondeterminism that is $T = \text{BNd}(T)$ where BNd , defined as follows

$$\text{BNd}(T) \triangleq \{\pi \in T \mid \forall \pi_1 \cdot \langle s, e \rangle \cdot \pi_2 = \pi \cdot |\{s' \mid \pi_1 \cdot \langle s, e \rangle \cdot \langle s', e' \rangle \cdot \pi'_2 \in T\}| \in \mathbb{N}\}, \quad (7)$$

meaning that for any state s encountered on a trace $\pi_1 \cdot \langle s, e \rangle \cdot \pi_2$ with next event e the number of possible successors s' on this and other possible traces $\pi_1 \cdot \langle s, e \rangle \cdot \langle s', e' \rangle \cdot \pi'_2$ with a prefix $\pi_1 \cdot \langle s, e \rangle$ is finite. Otherwise stated the number of possible successors s' of state s when event e occurs is finite.

Remark 1 (Locally versus globally bounded nondeterminism). Nondeterminism is local in (7), that is weakly bounded, since the maximum number of possible successive repetitions of an event e depends on the position of the event in the trace. This means that an event may take longer and longer to occur (e.g. after 1 then 2, 4, 8, ... computation steps) which seems compatible with the “numeric variant proof obligation rule” [4, Sect. 5.2.7]. For globally bounded nondeterminism, the stronger definition would be $\forall e \in E. \exists n \in \mathbb{N}. \forall \pi_1 \langle s, e \rangle \pi_2 = \pi \cdot |\{s' \mid \pi_1 \langle s, e \rangle \langle s', e' \rangle \pi'_2 \in T\}| < n$ where the same bound n on the number of possible consecutive occurrences of an event e is the same wherever this event occurs in an execution trace. $\square$

BNd in (7) is a lower closure operator (reductive, increasing, and idempotent).

$$\langle \mathbb{T}^F(S, E), \subseteq \rangle \xleftrightarrow[\text{BNd}]{1} \langle \mathbb{T}^{FB}(S, E), \subseteq \rangle \quad (8)$$

It is an abstraction of history independent traces onto $\mathbb{T}^{FB}(S, E) \triangleq \{T \in \mathbb{T}^\infty(S, E) \mid \text{Ffus}(T) = T \wedge \text{BNd}(T) = T\} = \text{BNd}(\text{Ffus}(\mathbb{T}^\infty(S, E)))$ (where $f(X) = \{f(x) \mid x \in X\} = \text{post}[f]X$ is the post image).

For simplicity we do not consider both cases of unbounded and bounded nondeterminism, as, for unbounded nondeterminism, it is sufficient to define $\text{BNd}(T) \triangleq T$ and to replace the naturals in variants by transfinite ordinals [20,23].

Composing (6) and (8) in the context $\langle S, E \rangle \in \wp(S) \times \wp(E)$, we get that the Event-B bounded semantics $\mathbb{T}^{FB}\langle S, E \rangle$ is an abstraction of the trace semantics $\mathbb{T}^\infty\langle S, E \rangle$,

$$\langle \mathbb{T}^\infty\langle S, E \rangle, \sqsubseteq \rangle \xleftrightarrow[\text{BNd} \circ \text{Ffus}]{1} \langle \mathbb{T}^{FB}\langle S, E \rangle, \sqsubseteq \rangle \quad (9)$$

2.8 Determinism. Determinism is the limit of bounded nondeterminism where each state has at most one possible successor. In that case, the semantics of the ultimate refinement should be deterministic, that is $T = \text{Det}(T)$ where

$$\text{Det}(T) \triangleq \{ \pi \in T \mid \forall \pi_1 \sqsubseteq \pi_2 = \pi . \forall \pi_1 \sqsubseteq \pi'_2 \in T . \pi_2 = \pi'_2 \} \quad (10)$$

which is a lower closure operator for $\sqsubseteq$. The deterministic semantics of the system is an abstraction of the trace semantics $\mathbb{T}^\infty\langle S, E \rangle$,

$$\langle \mathbb{T}^\infty\langle S, E \rangle, \sqsubseteq \rangle \xleftrightarrow[\text{Det} \circ \text{Ffus}]{1} \langle \mathbb{T}^{FD}\langle S, E \rangle, \sqsubseteq \rangle \quad (11)$$

where $\mathbb{T}^{FD}\langle S, E \rangle \triangleq \{ T \in \mathbb{T}^\infty\langle S, E \rangle \mid \text{Ffus}(T) = T \wedge \text{Det}(T) = T \} = \text{Det}(\text{Ffus}(\mathbb{T}^\infty\langle S, E \rangle))$.

2.9 Definition of Inevitability. Let $T \in \mathbb{T}^\infty\langle S, E \rangle$ be a set of traces. Let $P, Q \in \wp(S \times E)$ be trace state properties (defined in extension). Q is *inevitable* from P for T , written $\text{Inev}(P, T, Q)$, if and only if

$$\text{Inev}(P, T, Q) \triangleq \forall \pi \in T . \forall i \in \mathbb{N} . (\pi_i \in P) \Rightarrow (\exists j \geq i . \pi_j \in Q) \quad (12)$$

that is, whenever P holds during the computation, Q will always eventually hold later during this computation. A stronger requirement is that the distance $j - i$ be bounded for any trace π of T or, even stronger, the same upper bound on the distance $j - i$ may be required to hold for all traces in T .

2.10 Event Convergence. As a special case of inevitability, traces in T may be required to be *event-convergent*, meaning that no event can successively occur for ever. More precisely, no event can occur more than a bounded number of times.

$$\begin{aligned} \text{evt-cvg}(\pi) &\triangleq \forall e \in E, i \in \mathbb{N} . \exists n \in \mathbb{N} . \forall j > i \in \mathbb{N} . \\ &(\forall k \in [i, j] . \exists s' . \pi_k = \langle s', e \rangle) \Rightarrow (j - i < n) \end{aligned} \quad (13)$$

$$\text{Evt-Cvg}(T) \triangleq \{ \pi \in T \mid \text{evt-cvg}(\pi) \}$$

This is globally bounded nondeterminism, a local version $\forall e \in E . \exists n \in \mathbb{N} . \dots$ is also possible as observed in remark 1. Evt-Cvg is a lower closure operator for $\sqsubseteq$ [23, lem. 2.6.4 ~1]. Event-B may require sets of traces T to be event convergent, that is $\text{Evt-Cvg}(T) = T$. This is an abstraction onto

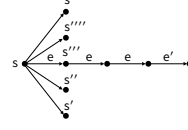
$$\mathbb{T}^{FBC}\langle S, E \rangle \triangleq \{ T \in \mathbb{T}^{FB}\langle S, E \rangle \mid \text{Evt-Cvg}(T) = T \} = \text{Evt-Cvg}(\mathbb{T}^{FB}\langle S, E \rangle), \quad (14)$$

as follows.

$$\langle \mathbb{T}^{FB}\langle S, E \rangle, \sqsubseteq \rangle \xleftrightarrow[\text{Evt-Cvg}]{1} \langle \mathbb{T}^{FBC}\langle S, E \rangle, \sqsubseteq \rangle \quad (15)$$

If the number $|\mathbf{E}|$ of possible events in $\mathbf{E}$ is finite¹, event convergence (15) implies that at any time in the computation any event will definitely occur later on.

Possible executions restricted by bounded nondeterminism (7) do limit the number of successors $s', \dots, s''''$ of a state s on the right figure, while (13) limits the number of consecutive occurrences of a given even e along an execution trace.



Only some events may be required to be convergent in which case the set $\mathbf{E}$ of all events should be replaced in the previous definitions by the subset $E \subseteq \mathbf{E}$ of events which must be convergent. For example in the location access controller [4, Ch. 16], it is possible that no one may ever enter the location. So the *pass* events (corresponding to a person going from a location to another) does not belong to E . This set E may even be empty, in which case *Evt-Cvg* is the identity.

Finally, convergence of events in E may not be guaranteed all along the refinement process. Therefore *Event-B* provides the convergence attribute *status* of an event with three possible values: *convergent* (the event must strictly decrease the variant), *anticipated* (the event must not increase the variant), and *ordinary* (the event may or may not not decrease the variant) [4, p. 183]. For simplicity, I only consider *convergent* and ignore the last two possibilities. Adding a *status* component to events and a case analysis would easily take into account all three possibilities.

3 Specifying the System Semantics by Abstraction Into Machines (i.e. Discrete Transition Systems)

Assuming that a set of states $\mathbf{S}$ has been defined by contexts [4, Sect. 5.1.3] and a set of event identifiers $\mathbf{E}$ is given [4, Sect. 5.1.7], we now study how a trace semantics $\mathbb{T}^{\text{EvB}}\langle \mathbf{S}, \mathbf{E} \rangle$ abstracting $\mathbb{T}^\infty\langle \mathbf{S}, \mathbf{E} \rangle$ can be defined in *Event-B* by machines [4, Sect. 5.1.5] that is transition systems (and possibly variants). Although any semantics $\mathbb{T}^\infty\langle \mathbf{S}, \mathbf{E} \rangle$ can be generated by a transition system (where states are a the pair of a trace and a position in that trace and transitions move forward the position in the trace), the history independent abstraction of Sect. 2.5 ensures that states of the transition system can be chosen as those states appearing on traces. So the abstraction *Ffus* is mandatory while the others *BNd*, *Det*, and *Evt-Cvg* are optional.

We consider the most demanding case by composing *Ffus* (6), *BNd* (8), and *Evt-Cvg* (15) in the context $\langle \mathbf{S}, \mathbf{E} \rangle \in \wp(\mathbf{S}) \times \wp(\mathbf{E})$ to get the *Event-B* semantics $\mathbb{T}^{\text{EvB}}\langle \mathbf{S}, \mathbf{E} \rangle \triangleq \mathbb{T}^{\text{FBC}}\langle \mathbf{S}, \mathbf{E} \rangle$ as an abstraction of the trace semantics $\mathbb{T}^\infty\langle \mathbf{S}, \mathbf{E} \rangle$, a characterization that also follows from [23, lem. 2.6.5~3]).

$$\frac{\langle \mathbb{T}^\infty\langle \mathbf{S}, \mathbf{E} \rangle, \sqsubseteq \rangle \xleftarrow[\text{Evt-Cvg} \circ \text{BNd} \circ \text{Ffus}]{1} \langle \mathbb{T}^{\text{EvB}}\langle \mathbf{S}, \mathbf{E} \rangle, \sqsubseteq \rangle}{(16)}$$

¹ In the number $|\mathbf{E}|$ of possible events in $\mathbf{E}$ is infinite then a new event can occur for ever. In *Event-B*, event must be declared as part of the machine structure and so are finite [4, Sect. 5.1.5].

3.1 Abstraction of a Trace Semantics to a Transition System. Given a trace semantics $T \in \mathbb{T}^{\text{EvB}}\langle S, E \rangle \subseteq \mathbb{T}^F\langle S, E \rangle$ of a system in Event-B, the first abstraction is a version $\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle$ of the classic abstraction of a set of traces into a discrete transition systems [23, def. 2.3:1], [14, Sect. 6] where inv is the set of reachable states, evts is the set of reachable events encountered on traces of the semantics, $\text{init} \subseteq \text{inv}$ is a set of initial states (abstracting the initial event from a universal initial state as used in Event-B), and $\tau(e)$ is the transition relation between a state and its immediate successor after occurrence of an event e .

$$\begin{aligned} \alpha_t\langle S, E \rangle(T) &\triangleq \langle \text{inv}, \text{evts}, \text{init}, \tau \rangle \in \mathbb{T}\langle S, E \rangle \quad \text{where} & (17) \\ \text{inv} &\triangleq \{s \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e \in E. \pi_i = \langle s, e \rangle\} \in \wp(S) \setminus \{\emptyset\} & (a) \\ \text{init} &\triangleq \{s \mid \exists \pi \in T. \exists e \in E. \pi_0 = \langle s, e \rangle\} \subseteq \text{inv} & (b) \\ \text{evts} &\triangleq \{e \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists s \in S. \pi_i = \langle s, e \rangle\} \in \wp(E) & (c) \\ \tau &\triangleq \lambda e \bullet \{ \langle s, s' \rangle \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e' \in E. \pi_i = \langle s, e \rangle \wedge & (d) \\ &\quad \pi_{i+1} = \langle s', e' \rangle \} \in \text{evts} \rightarrow \wp(\text{inv} \times \text{inv}) \end{aligned}$$

Observe that $T \in \mathbb{T}^{\text{FB}}\langle S, E \rangle$ implies the feasibility (since traces are infinite) and locally bounded nondeterminism requirements of Event-B (by (7))

$$\text{Feas}(\langle \text{inv}, \text{evts}, \tau \rangle) \triangleq \forall s \in \text{inv}. \exists s' \in \text{inv}, e \in \text{evts}. \langle s, s' \rangle \in \tau(e) \quad (18)$$

$$\text{BNd}(\langle \text{inv}, \text{evts}, \tau \rangle) \triangleq \forall s \in \text{inv}, e \in \text{evts}. |\{s' \mid \langle s, s' \rangle \in \tau(e)\}| \in \mathbb{N} \quad (19)$$

When applied to Event-B, Feas in (18) yields the feasibility proof obligation rule [4, Sect. 5.2.4] to ensure that no event can block the execution. For the globally bounded nondeterminism discussed in remark 1, we would have $\forall e \in \text{evts}. \exists n \in \mathbb{N}. \forall s \in \text{inv}. |\{s' \mid \langle s, s' \rangle \in \tau(e)\}| < n$.

This leads us to the definition of bounded feasible transition systems on states S and events E as

$$\begin{aligned} \mathbb{T}^{\text{FB}}\langle S, E \rangle &\triangleq \{ \langle \text{inv}, \text{evts}, \text{init}, \tau \rangle \in \mathbb{T}\langle S, E \rangle(T) \mid & (20) \\ &\quad \text{Feas}(\langle \text{inv}, \text{evts}, \tau \rangle) \wedge \text{BNd}(\langle \text{inv}, \text{evts}, \tau \rangle) \} \end{aligned}$$

The concretization, corresponding to [23, def. 2.4:1] and [4, Sect. 14.3.5, “Mathematical representation [of traces]”], is the set of traces generated by the transition system.

$$\begin{aligned} \gamma_t\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle) &\triangleq \{ \pi \in \mathbb{T}^\infty\langle S, E \rangle \mid \pi_0 \in \text{init} \times \text{evts} \wedge & (21) \\ &\quad \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \wedge \forall \langle s, e \rangle = \pi_i, \langle s', e' \rangle = \pi_{i+1}. \langle s, s' \rangle \in \tau(e) \} \end{aligned}$$

By $\text{init} \subseteq \text{inv}$ and the sound and complete induction principle for proving invariance [19, induction principle (i), p. 100], the invariance condition

$$\forall \pi \in \gamma_t\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle). \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \quad (22)$$

of (21) can be expressed as

$$\begin{aligned} \gamma_t\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle) &= \{ \pi \in \mathbb{L}^\infty\langle S, E \rangle \mid \pi_0 \in \text{init} \times \text{evts} \wedge \\ &\quad \forall i \in \mathbb{N}, \langle s, e \rangle \in \text{inv} \times \text{evts}, \langle s', e' \rangle \in S \times E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow \\ &\quad (\langle s, s' \rangle \in \tau(e) \wedge \langle s', e' \rangle \in \text{inv} \times \text{evts}) \} \end{aligned} \quad (23)$$

This requirement in (22) gives raise, when expressed as (23), to the invariant preservation proof obligation rule [4, Sect. 5.2.2] for a transition system.

This is a Galois connection mapping sets of traces to transition systems (where transition systems are ordered by componentwise inclusion $\subseteq$).

$$\langle \mathbb{T}^{\text{FB}}\langle S, E \rangle, \subseteq \rangle \xleftrightarrow[\alpha_t\langle S, E \rangle]{\gamma_t\langle S, E \rangle} \langle \mathbb{T}^{\text{rFB}}\langle S, E \rangle, \subseteq \rangle \quad (24)$$

3.2 Abstraction of a Trace Semantics to a Variant. Unfortunately, in the event convergence case $\mathbb{T}^{\text{EvB}}\langle S, E \rangle = \mathbb{T}^{\text{FBC}}\langle S, E \rangle$ in (15), the concretization $\gamma_t\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle)$ of a transition system $\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle$ may not belong to $\mathbb{T}^{\text{EvB}}\langle S, E \rangle \not\subseteq \mathbb{T}^\infty\langle S, E \rangle$ since the abstraction takes history independence and bounded nondeterminism into account, but forgets about event convergence.

To recover this missing information, we consider, in addition, the abstraction of event convergent traces into a variant function (ordered pointwise).

$$\langle T \in \mathbb{T}^{\text{EvB}}\langle S, E \rangle, \subseteq \rangle \xleftrightarrow[\alpha_v]{\gamma_v} \langle V\langle S, E, \tau \rangle, \leq \rangle \quad (25)$$

where the transition relation τ in (25) is given by

$$\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle = \alpha_t\langle S, E \rangle(T), \quad (26)$$

the abstract domain is

$$V\langle S, E, \tau \rangle \triangleq \{ V \in S \times E \rightarrow \mathbb{N} \mid \forall e \in E . \forall \langle s, s' \rangle \in \tau(e) . V\langle e, s' \rangle < V\langle e, s \rangle \}, \quad (27)$$

Notice that $V \in S \times E \rightarrow \mathbb{N}$ yields the numeric variant proof obligation rule [4, Sect. 5.2.7] (or the finite set variant proof obligation rule [4, Sect. 5.2.8] for variants in finite sets).

Finally, the abstraction is the maximal number of possible consecutive occurrences of a given event after reaching a state.

$$\begin{aligned} \alpha_v\langle S, E \rangle(T) &\triangleq \lambda \langle s, e \rangle . \max \{ j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T . \\ &\quad \forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle \} \end{aligned} \quad (28)$$

$$\gamma_v\langle S, E \rangle(V) \triangleq \{ \pi \in \mathbb{L}^\infty\langle S, E \rangle \mid \text{cvgt}\langle S, E \rangle(\pi, V) \} \quad (29)$$

$$\text{cvgt}\langle S, E \rangle(\pi, V) \triangleq \forall e \in E . \forall j > i \in \mathbb{N} .$$

$$(\forall k \in [i, j] . \exists s_k . \pi_k = \langle s_k, e \rangle) \Rightarrow (j - i < V\langle s_i, e \rangle)$$

where $\text{cvgt}\langle S, E \rangle(\pi, V)$ states that the number of consecutive occurrences of the same event e (possibly with different parameters) on trace π is bounded by the

variant V . (Instead of the naturals, Event-B also allows for variant functions in finite sets which we can consider as syntactic and semantic sugar.)

The codomain of α_v is $\mathbb{V}\langle S, E, \tau \rangle$, that is, by definition of α_v , the variant $V = \alpha_v\langle S, E \rangle$ strictly decreases after any transition τ of $\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle = \alpha_t\langle S, E \rangle(T)$ on the trace.

$$\forall e \in E. \forall \langle s, s' \rangle \in \tau(e). V\langle e, s' \rangle < V\langle e, s \rangle \quad (30)$$

Notice that this condition (30) on V in (31) yields the variant proof obligation rule [4, Sect. 5.2.9]. The proof that the codomain of γ_v is $\mathbb{T}^{\text{EvB}}\langle S, E \rangle$ directly follows from the well-foundedness of $\langle \mathbb{N}, \leq \rangle$. Finally, the pair $\langle \alpha_v, \gamma_v \rangle$ is the Galois connection (25).

3.3 Abstraction of a Trace Semantics to a Abstract Machine (Event convergent transition system). Since we require event convergence $\mathbb{T}^{\text{EvB}}\langle S, E \rangle = \mathbb{T}^{\text{FBC}}\langle S, E \rangle$, an abstract machine is an event convergent transition system.

$$\mathbb{T}^{\text{EvB}}\langle S, E \rangle \triangleq \{ \langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle \mid \langle \text{inv}, \text{evts}, \text{init}, \tau \rangle \in \mathbb{T}^{\text{FBC}}\langle S, E \rangle \wedge V \in \mathbb{V}\langle S, E, \tau \rangle \} \quad (31)$$

where event convergence is guaranteed by the variant V .

Let us calculate the abstraction of a trace-based definition semantics $T \in \mathbb{T}^{\text{EvB}}\langle S, E \rangle$ into an abstract machine (we define $f \dot{\times} g(x) \triangleq f(x) \times g(x)$ and $\langle x_1, \dots, x_n \rangle \times \langle y_1, \dots, y_m \rangle \triangleq \langle x_1, \dots, x_n, y_1, \dots, y_m \rangle$).

$$\begin{aligned} & \alpha_t \dot{\times} \alpha_v\langle S, E \rangle(T) \\ &= \langle \text{inv}, \text{init}, \text{evts}, \tau, V \rangle \quad \text{(def. (17) of } \alpha_t, \text{ (28) of } \alpha_v, \text{ and product)} \\ &= \{ \langle s \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e \in E. \pi_i = \langle s, e \rangle \rangle, \{ s \mid \exists \pi \in T. \exists e \in E. \pi_0 = \langle s, e \rangle \}, \{ e \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists s \in S. \pi_i = \langle s, e \rangle \}, \lambda e \cdot \{ \langle s, s' \rangle \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e' \in E. \pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle \}, \lambda \langle s, e \rangle \cdot \max\{j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T. \forall k \in [i, j]. \exists s_k. s_i = s \wedge \pi_k = \langle s_k, e \rangle \} \} \\ & \quad \text{(17.a)–(17.d) and (28)} \end{aligned} \quad (32)$$

Let us also calculate the later needed trace-based definition of the semantics of an abstract machine $\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle$ on $\langle S, E \rangle$, as follows.

$$\begin{aligned} & \gamma_t \times \gamma_v\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle) \\ &= \gamma_t\langle S, E \rangle(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle \cap \gamma_v\langle S, E \rangle(V)) \quad \text{(def. product } \times \text{)} \\ &= \{ \pi \in \mathbb{T}^\infty\langle S, E \rangle \mid \pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \wedge \forall \langle s, e \rangle = \pi_i, \langle s', e' \rangle = \pi_{i+1}. \langle s, s' \rangle \in \tau(e) \} \cap \{ \pi \in \mathbb{T}^\infty\langle S, E \rangle \mid \text{cvgt}\langle S, E \rangle(\pi, V) \} \quad \text{(def. (21) of } \gamma_t \text{ and (29) of } \gamma_v) \\ &= \{ \pi \in \mathbb{T}^\infty\langle S, E \rangle \mid \pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \wedge \forall \langle s, e \rangle = \pi_i, \langle s', e' \rangle = \pi_{i+1}. \langle s, s' \rangle \in \tau(e) \wedge \text{cvgt}\langle S, E \rangle(\pi, V) \} \quad \text{(def. } \cap \text{)} \end{aligned} \quad (33)$$

In words, a trace is infinite, must start with an initial state in init and satisfy the invariant $\text{inv} \times \text{evts}$, successive states on the trace for a given event e must be in the transition relation $\langle s, s' \rangle \in \tau(e)$ for that event e , and the number of consecutive occurrences of the same event e is bounded by the variant V .

Thanks to history independence, we now have the Galois isomorphism (where $\langle x_1, \dots, x_n, y_1, \dots, y_m \rangle \dot{\subseteq} \times \dot{\subseteq} \langle x'_1, \dots, x'_n, y'_1, \dots, y'_m \rangle \triangleq \langle x_1, \dots, x_n \rangle \dot{\subseteq} \langle x'_1, \dots, x'_n \rangle \wedge \langle y_1, \dots, y_m \rangle \dot{\subseteq} \langle y'_1, \dots, y'_m \rangle = x_1 \subseteq x'_1 \wedge \dots \wedge x_n \subseteq x'_n \wedge y_1 \subseteq y'_1 \wedge \dots \wedge y_m \subseteq y'_m$ componentwise)

$$\langle \mathbb{T}^{\text{EvB}}(S, E), \subseteq \rangle \xleftrightarrow[\alpha_t \dot{\times} \alpha_v(S, E)]{\gamma_t \dot{\times} \gamma_v(S, E)} \langle \mathbb{T}^{\text{EvB}}(S, E), \dot{\subseteq} \times \dot{\subseteq} \rangle \quad (34)$$

3.4 On the Definition of a Trace Semantics of Abstract Machines in Event-B. The Galois isomorphism (34) proves that the Event-B method of defining a history-independent, event convergent, bounded trace semantics (14) in $\mathbb{T}^{\text{EvB}}(S, E)$ by means of a transition system and a variant $\mathbb{T}^{\text{EvB}}(S, E)$ (31) is sound and complete.

Recall that the definition of machines in Event-B defines proof obligations to ensure that S and E are well defined and the specified transition system $\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle$ and variant V belong to $\mathbb{T}^{\text{EvB}}(S, E) \times \mathbb{V}(S, E, \tau)$ which ensure invariant initialization and preservation, deadlock freeness, event convergence, feasibility, and bounded nondeterminism of the trace semantics. History independence directly follows from the fact that traces are generated by the transition system, as defined by $\gamma_t(S, E)(\langle \text{inv}, \text{evts}, \text{init}, \tau \rangle)$ (formalizing e.g. context extension).

In what follows, we will need a definition of the reflexive transitive closure τ^* of the transition relation τ , as follows ($n \in \mathbb{N}$).

$$\begin{aligned} \tau^n &\triangleq \{ \langle s, s' \rangle \mid \exists s_0, e_0, s_1, e_1, \dots, s_n . s_0 = s \wedge \forall k \in [0, n[. \langle s_k, s_{k+1} \rangle \in \tau(e_k) \wedge s_n = s' \} \\ \tau^* &\triangleq \bigcup_{n \in \mathbb{N}} \tau^n \end{aligned} \quad (35)$$

We write $\tau[e]^k$ and $\tau[e]^*$ for the case when all events $e_k, k \in [0, n[$ are restricted to be e .

4 Refinement of System Trace Semantics

The definition of system semantics by machines is only an aspect of Event-B. The main technique in Event-B is the development by successive refinements.

Whereas abstraction interpretation establishes a connection between different semantics (such as set of traces in operational semantics and local invariants in Hoare logic)², refinements establishes a connection between semantics of the same nature (i.e. traces, maybe on different states). This is not a fundamental difference since the concretization can always be used, at least in theory, to reason in the concrete (i.e. set of traces so that the abstraction becomes a closure [18, Sect. 5.2]), although this is not done in practice since set of traces are not directly implementable in a machine. The main difference is that refinement

² Since Event-B is only concerned with machines, we only need to consider in this chapter the abstract interpretation of machines by machines (more precisely their trace semantics).

aims at providing the implementation of one trace, whereas abstract interpretation aims at providing the implementation of a set of traces, or a machine implementation of it (such as local invariants in deductive verification or abstract domains in static analysis). Nevertheless, the reasoning in programming by refinement and abstract interpretation are quite similar since in both cases one works by reference to a specification (which can always be represented as a set of traces).

4.1 Simple Trace Refinement. In the simplest case, a refinement $T \in \mathbb{T}^{\text{FB}}\langle S, E \rangle$ of $\bar{T} \in \mathbb{T}^{\text{FB}}\langle S, E \rangle$ in Event-B eliminates some possible executions as specified by $S \in \mathbb{T}^{\text{FB}}\langle S, E \rangle$. So T is more implementable than $\bar{T}$ since some possible executions of $\bar{T}$ have been eliminated in T . Define $\alpha_S(X) \triangleq X \cap S = X \setminus \neg S$ and $\gamma_S(Y) \triangleq Y \cup \neg S$ so that we have $T = \alpha_S(\bar{T})$. This is a classic exclusion abstraction $\langle \mathbb{T}^{\text{FB}}\langle S, E \rangle, \subseteq \rangle \xrightarrow[\alpha_S]{\gamma_S} \langle \mathbb{T}^{\text{FB}}\langle S, E \rangle, \subseteq \rangle$ [15, Ex. 11.5]. It follows that the refined system semantics is a subset of the abstract one

$$\emptyset \neq T \subseteq \bar{T}. \quad (36)$$

(i.e. “Any trace of a refined model must also be a trace of the abstraction” [4, Sect. 14.4.3]).

When T and $\bar{T}$ are respectively defined by machines $t = \langle \text{inv}, \text{evts}, \text{init}, \tau \rangle$ and $\bar{t} = \langle \overline{\text{inv}}, \overline{\text{evts}}, \overline{\text{init}}, \bar{\tau} \rangle$, that is $T = \gamma_t(S, E)(t)$ and $\bar{T} = \gamma_{\bar{t}}(S, E)(\bar{t})$, simple trace refinement can be expressed in term of these machines as

$$\emptyset \neq t \subseteq \bar{t} \quad (37)$$

stating that “a refined trace is also an abstract trace”. Because we consider only feasible transition systems (18), the requirement “we do not want a refined trace to be unable to be extended if the corresponding abstract one is able to be extended” [4, (I), p. 428] and [4, (II), p. 429] is always satisfied.

4.2 Duality Between Refinement and Abstract Interpretation. The previous refinement α_S eliminates undesirable execution in $\neg S$. In abstract interpretation, the abstraction α_S ignores all information about the eliminated executions in $\neg S$. The abstract semantics is therefore a subset of the concrete one. This yields the following confusing vocabulary clash

	$T = \alpha_S(\bar{T})$	$\bar{T} = \gamma_S(T)$	
Event-B	T refines $\bar{T}$	$\bar{T}$ abstracts T	(38)
Abstract interpretation	T abstracts $\bar{T}$	$\bar{T}$ concretizes T	

Notice that this duality also appears in the definition of the word “abstraction” in dictionaries as “the process of considering something independently of its associations, attributes, or concrete accompaniments” as in abstract interpretation or “the process of removing something” as in refinement ³.

³ Apple Dictionary, Version 2.3.0 (294).

Finally, the order dual $\langle C, \supseteq \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \supseteq \rangle$ of a Galois connection $\langle C, \subseteq \rangle \xleftrightarrow[\alpha]{\gamma}$ $\langle A, \subseteq \rangle$ is a Galois connection $\langle A, \subseteq \rangle \xleftrightarrow[\gamma]{\alpha} \langle C, \subseteq \rangle$ where α and γ is exchanged [15, Lem. 11.25] showing the ambiguity of the formulations “abstraction”, “refinement”, and “concretization” which heavily depends on the considered ordering and are exchanged for dual orderings.

4.3 Homomorphic Trace Refinement. The requirement in simple trace refinement that concrete and abstract states and events be the same can be lifted by considering an homomorphic abstraction [15, Ex. 11.6] defined by $h \in S \times E \rightarrow \bar{S} \times \bar{E}$ as

$$\alpha_h(T) \triangleq \{h(\pi) \mid \pi \in T\} \quad \text{where} \quad h(\pi) \triangleq \lambda i \in \mathbb{N} \cdot h(\pi_i) \quad (39)$$

so that $\langle \mathbb{T}^{\text{EvB}}(S, E), \subseteq \rangle \xleftrightarrow[\alpha_h]{\gamma_h} \langle \mathbb{T}^{\text{EvB}}(\bar{S}, \bar{E}), \subseteq \rangle$. Homomorphic trace refinement is then $\emptyset \neq \alpha_h(T) \subseteq \bar{T}$ which can be easily translated to machines.

4.4 External Trace Refinement. In case trace transitions are not directly observable, we let $\mathbf{Es}$ and $\bar{\mathbf{Es}}$ be (external observation) sets, $f \in S \times E \rightarrow \mathbf{Es}$ and $\bar{f} \in \bar{S} \times \bar{E} \rightarrow \bar{\mathbf{Es}}$ be observation functions, and define the homomorphic abstractions $\alpha_f(T) \triangleq \{f(\pi) \mid \pi \in T\}$ where $f(\pi) \triangleq \lambda i \in \mathbb{N} \cdot f(\pi_i)$ and $\alpha_{\bar{f}}(\bar{T}) \triangleq \{\bar{f}(\bar{\pi}) \mid \bar{\pi} \in \bar{T}\}$ where $\bar{f}(\bar{\pi}) \triangleq \lambda i \in \mathbb{N} \cdot \bar{f}(\bar{\pi}_i)$. Given $h \in \mathbf{Es} \rightarrow \bar{\mathbf{Es}}$, external trace refinement is defined as $\emptyset \neq \alpha_h(\alpha_f(T)) \subseteq \alpha_{\bar{f}}(\bar{T})$. This is extended to machines to get [4, (IV), p. 433].

4.5 Total Simulation Refinement. Let $\rho \in (S \times E) \rightarrow \wp(\bar{S} \times \bar{E})$ be a total binary relation from concrete to abstract transition states. Define the abstraction

$$\alpha_\rho(T) \triangleq \{\lambda i \in \mathbb{N} \cdot \bar{\pi}_i \mid \pi \in T \wedge \forall i \in \mathbb{N} . \bar{\pi}_i \in \rho(\pi_i)\} \quad (40)$$

(which is (39) when $\rho = h$) such that $\langle \mathbb{T}^{\text{EvB}}(S, E), \subseteq \rangle \xleftrightarrow[\alpha_\rho]{\gamma_\rho} \langle \mathbb{T}^{\text{EvB}}(\bar{S}, \bar{E}), \subseteq \rangle$. We can define forward simulation refinement as $\emptyset \neq \alpha_\rho(T) \subseteq \bar{T}$.

4.6 Forward Simulation Refinement. Given $f \in S \times E \rightarrow \mathbf{Es}$, $\bar{f} \in \bar{S} \times \bar{E} \rightarrow \bar{\mathbf{Es}}$, and $h \in S \times E \rightarrow \bar{S} \times \bar{E}$, conditions [4, (C1), p. 434], [4, (C2) p. 435], and [4, (C3), p. 435] imply that the forward simulation refinement is an external trace refinement. This translates to machines to get the rules of [4, Sect. 15.5.3].

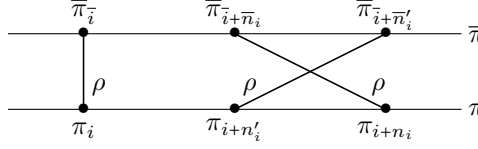
4.7 Backward Simulation Refinement. Considering [4, (C2') p. 439], we get backward simulation refinement,

4.8 Trace Refinement [4, Sect. 14.6]. Assume that $\pi \in \mathbb{T}^\infty(S, E)$, $\bar{\pi} \in \mathbb{T}^\infty(\bar{S}, \bar{E})$, and $\rho \in \wp((S \times E) \times (\bar{S} \times \bar{E}))$ is a (so-called simulation) relation (which may be restricted to be functional). ρ defines a relation between concrete and abstract states and events (as specifying in Event-B by *gluing invariants* for states [4, p. 181] and *witnesses* for variables and parameters of corresponding events [4, p. 183]).

A trace π is a *refinement* of $\bar{\pi}$ by ρ , denoted $R_\rho(\pi, \bar{\pi})$, if and only if

$$\begin{aligned} R_\rho(\pi, \bar{\pi}) \triangleq & \exists \eta > 0, i_0, \bar{i}_0 \in \mathbb{N} . \langle \pi_{i_0}, \bar{\pi}_{\bar{i}_0} \rangle \in \rho \wedge \\ & \forall i, \bar{i} \in \mathbb{N} . (\langle \pi_i, \bar{\pi}_{\bar{i}} \rangle \in \rho) \Rightarrow (\exists n_i, \bar{n}_{\bar{i}} \in [0, \eta] . n_i + \bar{n}_{\bar{i}} > 0 \wedge \langle \pi_{i+n_i}, \bar{\pi}_{\bar{i}+\bar{n}_{\bar{i}}} \rangle \in \rho) \end{aligned} \quad (41)$$

Ultimately, some initial transitions π_{i_0} and $\bar{\pi}_{\bar{i}_0}$ must be related by ρ (one can also require the stronger $i_0 = \bar{i}_0 = 0$). Later on, if two transitions π_i and $\bar{\pi}_{\bar{i}}$ are related by ρ then farther concrete and abstract transition π_{i+n_i} and $\bar{\pi}_{\bar{i}+\bar{n}_{\bar{i}}}$ must be related by ρ , and this for ever. If $\bar{n}_{\bar{i}} = 0$ they are split [4, Sect. 14.6.1] and if $n_i = 0$ then consecutive abstract events are merged ([4, Sect. 14.6.2] is for alternative events merging, also allowed by (41)). Notice that n_i and $\bar{n}_{\bar{i}}$ need not be unique as in



To avoid refinements taking larger and larger number of computation steps, they are bounded by η (depending on π and $\bar{\pi}$). It follows that an abstract computation step can be refined by bounded finitely many concrete computation steps or only bounded finitely many consecutive abstract transitions may disappear.

4.9 Trace Semantics Refinement. Let us define the refinement abstraction $\alpha_\rho(\mathbf{T})$ of $\mathbf{T} \in \mathbb{T}^\infty(\mathbf{S}, \mathbf{E})$ as

$$\alpha_\rho(\mathbf{T}) \triangleq \text{post}[R_\rho]\mathbf{T} = \{ \bar{\pi} \in \mathbb{T}^\infty(\bar{\mathbf{S}}, \bar{\mathbf{E}}) \mid \exists \pi \in \mathbf{T} . R_\rho(\pi, \bar{\pi}) \} \quad (42)$$

This is a Galois connection [15, (12.22)]

$$\langle \mathbb{T}^\infty(\mathbf{S}, \mathbf{E}), \subseteq \rangle \xleftrightarrow[\alpha_\rho]{\gamma_\rho} \langle \mathbb{T}^\infty(\bar{\mathbf{S}}, \bar{\mathbf{E}}), \subseteq \rangle \quad (43)$$

where

$$\gamma_\rho(\bar{\mathbf{T}}) \triangleq \text{pre}[R_\rho]\bar{\mathbf{T}} = \{ \pi \in \mathbb{T}^\infty(\mathbf{S}, \mathbf{E}) \mid \forall \bar{\pi} . R_\rho(\pi, \bar{\pi}) \Rightarrow (\bar{\pi} \in \bar{\mathbf{T}}) \} \quad (44)$$

We say that $\bar{\mathbf{T}} \in \mathbb{T}^\infty(\bar{\mathbf{S}}, \bar{\mathbf{E}})$ is an abstraction of $\mathbf{T} \in \mathbb{T}^\infty(\mathbf{S}, \mathbf{E})$ (or $\mathbf{T}$ is a refinement of $\bar{\mathbf{T}}$) if and only if $R_\rho(\mathbf{T}, \bar{\mathbf{T}})$ holds, where

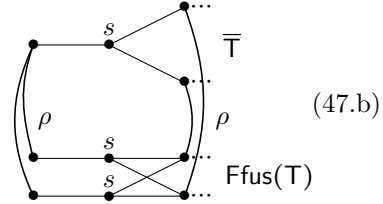
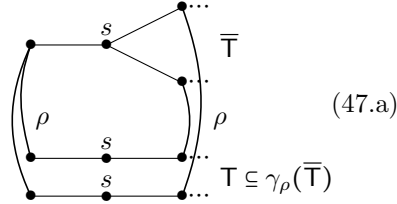
$$\bar{R}_\rho(\mathbf{T}, \bar{\mathbf{T}}) \triangleq \alpha_\rho(\mathbf{T}) \subseteq \bar{\mathbf{T}} \Leftrightarrow \mathbf{T} \subseteq \gamma_\rho(\bar{\mathbf{T}}) \quad (45)$$

which is equivalent to

$$\begin{aligned} \forall \pi \in \mathbf{T} . \exists \bar{\pi} \in \bar{\mathbf{T}} . \exists \eta > 0, i_0, \bar{i}_0 \in \mathbb{N} . \langle \pi_{i_0}, \bar{\pi}_{\bar{i}_0} \rangle \in \rho \wedge \\ \forall i, \bar{i} \in \mathbb{N} . (\langle \pi_i, \bar{\pi}_{\bar{i}} \rangle \in \rho) \Rightarrow (\exists n_i, \bar{n}_{\bar{i}} \in [1, \eta] . \langle \pi_{i+n_i}, \bar{\pi}_{\bar{i}+\bar{n}_{\bar{i}}} \rangle \in \rho) \end{aligned} \quad (46)$$

so that no concrete trace can be unrelated to an abstract trace (although the opposite is possible since some abstract behaviors may disappear in the concrete).

4.10 History Independence Is Not Necessarily Preserved by Trace Refinement. Let $\bar{T} \in \mathbb{T}^{\text{EvB}}(\bar{S}, \bar{E})$ be an abstract Event-B semantics (hence history independent). Let T be a refinement of $\bar{T}$ so that $T \subseteq \gamma_\rho(\bar{T})$ by (45). The following counter-example (47.a) shows that T may not be a concrete Event-B semantics (since it is not history independent, that is $\text{Ffus}(\gamma_\rho(\bar{T})) \neq \gamma_\rho(\bar{T})$) although $\text{Ffus}(\bar{T}) = \bar{T}$ as seen in (47.b)).



Therefore, I define $T \in \mathbb{T}^\infty(S, E)$ to be a refinement of $\bar{T} \in \mathbb{T}^{\text{EvB}}(\bar{S}, \bar{E})$ for $\rho \in \wp((S \times E) \times (\bar{S} \times \bar{E}))$ if and only if $T \in \mathbb{T}^F(S, E) \wedge R_\rho(T, \bar{T})$. This condition is naturally satisfied in [4, Sect. 14.6] since Jean-Raymond defines refinement for transition systems whereas I infer it by abstraction of trace refinement.

5 Refinement of Machines

Given an abstract event convergent transition system $\bar{t} = \langle \bar{\text{inv}}, \bar{\text{evts}}, \bar{\text{init}}, \bar{\tau}, \bar{V} \rangle \in \mathbb{T}^{\text{EvB}}(\bar{S}, \bar{E})$ generating the abstract semantics $\bar{T} = \gamma_t \times \gamma_v(\bar{t})$ and a concrete event convergent transition system $t = \langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle \in \mathbb{T}^{\text{EvB}}(S, E)$ generating the concrete semantics $T = \gamma_t \times \gamma_v(t)$, we look for a sufficient (and preferably necessary) condition on $\bar{t}$ and t ensuring that $\bar{R}_\rho(T, \bar{T})$. We proceed by calculational design. We must have

$$\begin{aligned}
& \bar{R}_\rho(T, \bar{T}) \\
& \Leftrightarrow \bar{R}_\rho(\gamma_t \times \gamma_v(t), \gamma_t \times \gamma_v(\bar{t})) && \{\text{def. } T \text{ and } \bar{T}\} \\
& \Leftrightarrow \bar{R}_\rho(\gamma_t \times \gamma_v(\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle), \gamma_t \times \gamma_v(\langle \bar{\text{inv}}, \bar{\text{evts}}, \bar{\text{init}}, \bar{\tau}, \bar{V} \rangle)) && \{\text{def. } t \text{ and } \bar{t}\} \\
& \Leftrightarrow \alpha_\rho(\gamma_t \times \gamma_v(\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle)) \subseteq \gamma_t \times \gamma_v(\langle \bar{\text{inv}}, \bar{\text{evts}}, \bar{\text{init}}, \bar{\tau}, \bar{V} \rangle) && \{\text{def. (45) of } \bar{R}_\rho\} \\
& \Leftrightarrow \{\bar{\pi} \mid \exists \pi \in \gamma_t \times \gamma_v(\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle) . \bar{R}_\rho(\pi, \bar{\pi})\} \subseteq \gamma_t \times \gamma_v(\langle \bar{\text{inv}}, \bar{\text{evts}}, \bar{\text{init}}, \bar{\tau}, \bar{V} \rangle) && \{\text{def. (42) of } \alpha_\rho\} \\
& \Leftrightarrow \forall \bar{\pi} . \forall \pi \in \gamma_t \times \gamma_v(\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle) . \bar{R}_\rho(\pi, \bar{\pi}) \Rightarrow (\bar{\pi} \in \gamma_t \times \gamma_v(\langle \bar{\text{inv}}, \bar{\text{evts}}, \bar{\text{init}}, \bar{\tau}, \bar{V} \rangle)) && \{\text{def. } \subseteq\} \\
& \Leftrightarrow \forall \bar{\pi} . \forall \pi \in \{\pi \in \mathbb{T}^\infty(S, E) \mid \pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}, s, s' \in \text{inv}, e, e' \in E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow ((s, s') \in \tau(e)) \wedge \text{cvg}(\bar{S}, \bar{E})(\pi, \bar{V}))\} . \\
& \quad \bar{R}_\rho(\pi, \bar{\pi}) \Rightarrow \\
& \quad (\bar{\pi} \in \{\bar{\pi} \in \mathbb{T}^\infty(\bar{S}, \bar{E}) \mid \bar{\pi}_0 \in \bar{\text{init}} \times \bar{\text{evts}} \wedge \forall i \in \mathbb{N}, \bar{s}, \bar{s}' \in \bar{\text{inv}}, \bar{e}, \bar{e}' \in \bar{E} . (\bar{\pi}_i = \langle \bar{s}, \bar{e} \rangle \wedge \bar{\pi}_{i+1} = \langle \bar{s}', \bar{e}' \rangle) \Rightarrow ((\bar{s}, \bar{s}') \in \bar{\tau}(\bar{e})) \wedge \text{cvg}(\bar{S}, \bar{E})(\bar{\pi}, \bar{V}))\}) \\
& \quad \{\text{characterization (33) of } \gamma_t \times \gamma_v\}
\end{aligned}$$

$$\begin{aligned}
&\Leftrightarrow \forall \bar{\pi}, \pi . (\pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}, s, s' \in \text{inv}, e, e' \in E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow (\langle s, s' \rangle \in \tau(e)) \wedge \text{cvgt}(\langle S, E \rangle(\pi, V) \wedge \bar{R}_\rho(\pi, \bar{\pi})) \Rightarrow \\
&\quad (\bar{\pi} \in \mathbb{L}^\infty(\bar{S}, \bar{E}) \mid \bar{\pi}_0 \in \overline{\text{init}} \times \overline{\text{evts}} \wedge \forall i \in \mathbb{N}, \bar{s}, \bar{s}' \in \overline{\text{inv}}, \bar{e}, \bar{e}' \in \bar{E} . (\bar{\pi}_i = \langle \bar{s}, \bar{e} \rangle \wedge \bar{\pi}_{i+1} = \langle \bar{s}', \bar{e}' \rangle) \Rightarrow (\langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}(\bar{e})) \wedge \text{cvgt}(\bar{S}, \bar{E})(\bar{\pi}, \bar{V})) \quad (\text{def. } \in) \\
&\Leftrightarrow \forall \bar{\pi}, \pi . (\pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}, s, s' \in \text{inv}, e, e' \in E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow (\langle s, s' \rangle \in \tau(e)) \wedge \text{cvgt}(\langle S, E \rangle(\pi, V) \wedge (\exists \eta > 0, i_0, \bar{i}_0 \in \mathbb{N} . \langle \pi_{i_0}, \bar{\pi}_{\bar{i}_0} \rangle \in \rho \wedge \forall i, \bar{i} \in \mathbb{N} . (\langle \pi_i, \bar{\pi}_{\bar{i}} \rangle \in \rho) \Rightarrow (\exists n_i, \bar{n}_{\bar{i}} \in [0, \eta] . n_i + \bar{n}_{\bar{i}} > 0 \wedge \langle \pi_{i+n_i}, \bar{\pi}_{\bar{i}+\bar{n}_{\bar{i}}} \rangle \in \rho))) \Rightarrow \\
&\quad (\bar{\pi} \in \mathbb{L}^\infty(\bar{S}, \bar{E}) \mid \bar{\pi}_0 \in \overline{\text{init}} \times \overline{\text{evts}} \wedge \forall i \in \mathbb{N}, \bar{s}, \bar{s}' \in \overline{\text{inv}}, \bar{e}, \bar{e}' \in \bar{E} . (\bar{\pi}_i = \langle \bar{s}, \bar{e} \rangle \wedge \bar{\pi}_{i+1} = \langle \bar{s}', \bar{e}' \rangle) \Rightarrow (\langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}(\bar{e})) \wedge \text{cvgt}(\bar{S}, \bar{E})(\bar{\pi}, \bar{V})) \quad (\text{def. (41) of } \bar{R}_\rho(\pi, \bar{\pi})) \\
&\Leftrightarrow \forall \bar{\pi}, \pi . ((\pi_0 \in \text{init} \times \text{evts} \wedge \exists i_0 \in \mathbb{N} . \forall i \in [0, i_0[, s, s' \in \text{inv}, e, e' \in E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow (\langle s, s' \rangle \in \tau(e)) \wedge \langle \pi_{i_0}, \bar{\pi}_0 \rangle \in \rho) \Rightarrow (\bar{\pi}_0 \in \overline{\text{init}})) \wedge \quad (47) \\
&\quad (A)
\end{aligned}$$

$$\begin{aligned}
&((\forall i \in \mathbb{N}, s, s' \in \text{inv}, e, e' \in E . (\pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle) \Rightarrow (\langle s, s' \rangle \in \tau(e)) \wedge \text{cvgt}(\langle S, E \rangle(\pi, V) \wedge \exists \eta > 0 . \forall i, \bar{i} \in \mathbb{N} . (\langle \pi_i, \bar{\pi}_{\bar{i}} \rangle \in \rho) \Rightarrow (\exists j \in [i, i + \eta] . \langle \pi_j, \bar{\pi}_{\bar{j}+1} \rangle \in \rho)) \Rightarrow \\
&(\forall i \in \mathbb{N}, \bar{s}, \bar{s}' \in \overline{\text{inv}}, \bar{e}, \bar{e}' \in \bar{E} . (\bar{\pi}_i = \langle \bar{s}, \bar{e} \rangle \wedge \bar{\pi}_{i+1} = \langle \bar{s}', \bar{e}' \rangle) \Rightarrow (\langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}(\bar{e})) \wedge \text{cvgt}(\bar{S}, \bar{E})(\bar{\pi}, \bar{V})) \quad (B)
\end{aligned} \quad (48)$$

(A) separates the initial states from the later trace transitions considered in (B). This is correct, since:

($\Leftarrow$) follows from $P \Rightarrow Q$ and $R \Rightarrow S$ implies $P \wedge R \Rightarrow Q \wedge S$.

($\Rightarrow$) the converse might not hold e.g. in case there is no infinite trace starting from the initial state π_0 or $\bar{\pi}_0$. However, this case is prevented by $\text{init} \subseteq \text{inv}$ and $\overline{\text{init}} \subseteq \overline{\text{inv}}$ in (20) as well as feasibility (18). If (A) holds they imply that there is at least one concrete and abstract traces starting from these initial states and then (B) concludes the implication for all of these traces.)

(A)

$$\Leftrightarrow \forall s_0 \in \text{init} . \exists s . \langle s_0, s \rangle \in \tau^* \wedge \exists \bar{s}_0 \in \overline{\text{init}} . \langle s, \bar{s}_0 \rangle \in \rho$$

This is a slight generalization of the base condition in the original definition of simulation [32] where $s_0 = s$.

(B)

$$\begin{aligned}
&\Leftrightarrow \forall \langle s, e \rangle \in \text{post}[\tau^*](\text{init} \times \text{evts}), \forall \langle \bar{s}, \bar{e} \rangle \in \text{post}[\bar{\tau}^*](\overline{\text{init}} \times \overline{\text{evts}}), \langle s', e' \rangle . ((\langle s, e \rangle, \langle \bar{s}, \bar{e} \rangle) \in \rho \wedge \langle s, s' \rangle \in \tau(e) \wedge \text{cvgt}(\langle S, E \rangle(\text{inv}, \text{init}, \text{evts}, \tau, V)) \Rightarrow (\exists \langle \bar{s}', \bar{e}' \rangle . \langle \langle s', e' \rangle, \langle \bar{s}', \bar{e}' \rangle \rangle \in \rho \wedge \langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}^*(\bar{e}) \wedge \text{cvgt}(\bar{S}, \bar{E})(\overline{\text{inv}}, \overline{\text{init}}, \overline{\text{evts}}, \bar{\tau}, \bar{V}))
\end{aligned}$$

where

$$\begin{aligned}
&\text{cvgt}(\langle S, E \rangle(\text{inv}, \text{init}, \text{evts}, \tau, V)) \triangleq \forall e \in E, \langle s, e \rangle \in \text{inv} \cap \text{post}[\tau^*](\text{init} \times \text{evts}), \\
&\quad k \in \mathbb{N}, \langle s', e \rangle \in \text{inv} \cap \text{post}[\tau[e]^k](\langle s, e \rangle) . k \leq V\langle s, e \rangle
\end{aligned} \quad (49)$$

expressing that any reachable state cannot be followed by more e -transitions than allowed by the variant V .

{The equivalence follows from the fact that π and $\bar{\pi}$ satisfy (B) implies that π_i is a descendant of the initial states and there is a transition between π_i and π_{i+1} by $\pi \in \mathbb{T}^{\text{EvB}}\langle S, E \rangle$ and def. **post** and same for $\bar{\pi}$ and the simulation relation ρ holds. Conversely, $\langle \bar{s}', \bar{e}' \rangle$ and $\langle s', e \rangle \in \text{inv} \cap \text{post}[\tau[e]^k]\{\langle s, e \rangle\}$ imply the existence of π by def. **post**, τ^* , and $\tau[e]^k$. Same for $\bar{\pi}$.

The reasoning is the same for convergence since **post** $[\tau^*]$ is equivalent to the existence of an infinite trace by def. transition system satisfying feasibility (18).}

$$\Leftrightarrow \exists I, \bar{I}, \forall \langle s, e \rangle, \langle \bar{s}, \bar{e} \rangle, \langle s', e' \rangle . ((\text{init} \times \text{evts}) \subseteq I \wedge \text{post}[\tau](I) \subseteq I \wedge (\overline{\text{init}} \times \text{evts}) \subseteq \bar{I} \wedge \text{post}[\bar{\tau}](\bar{I}) \subseteq \bar{I} \wedge \langle s, e \rangle \in I, \langle \bar{s}, \bar{e} \rangle \in \bar{I} \wedge \langle \langle s, e \rangle, \langle \bar{s}, \bar{e} \rangle \rangle \in \rho \wedge \langle s, s' \rangle \in \tau(e) \wedge \text{cvgt}(S, E)(\text{inv}, \text{init}, \text{evts}, \tau, V)) \Rightarrow (\exists \langle \bar{s}', \bar{e}' \rangle . \langle \langle s', e' \rangle, \langle \bar{s}', \bar{e}' \rangle \rangle \in \rho \wedge \langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}^*(\bar{e}) \wedge \text{cvgt}(\bar{S}, \bar{E})(\overline{\text{inv}}, \overline{\text{init}}, \overline{\text{evts}}, \bar{\tau}, \bar{V})) \quad (50)$$

{By the Turing/Naur/Floyd induction principle, we have $\text{post}[\tau^*](\text{init} \times \text{evts}) \subseteq I$ and $\text{post}[\bar{\tau}^*](\overline{\text{init}} \times \overline{\text{evts}}) \subseteq \bar{I}$;
 $(\Rightarrow)$ Take $I = \text{post}[\tau^*](\text{init} \times \text{evts})$ and $\bar{I} = \text{post}[\bar{\tau}^*](\overline{\text{init}} \times \overline{\text{evts}})$;
 $(\Leftarrow)$ If the property holds for all elements of I and $\bar{I}$, it holds for all elements of any of their subsets}

Verification condition (50) is split into several proof obligations in Event-B where I and $\bar{I}$ are machine invariants (according to the invariant preservation proof obligation rule **INV** [4, Sect. 5.2.2]).

(50) “ensures that when a concrete event is enabled, so is the corresponding abstract one” [4, p. 192], which is proof obligation **GRD** of [4, Sect. 5.2.4].

Assume that in (50), we have $\langle \langle s, e \rangle, \langle \bar{s}, \bar{e} \rangle \rangle \in \rho \wedge \langle \langle s, e \rangle, \langle \bar{s}, \bar{e}' \rangle \rangle \in \rho \wedge \langle s, s' \rangle \in \tau(e)$ so that the concrete event merges two different abstract events, a case considered in [4, Sect. 5.2.5]. Then (50) states that for each of these two abstract events a transition must be possible. This is precisely the guard merging proof obligation rule **MRG** in [4, Sect. 5.2.5]. Notice that **MRG** is restricted to finitely possible abstract events while (50) doesn’t. However, the events are finite in Event-B, so this is equivalent.

In case $\langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}^*(\bar{e})$ in (50) is restricted to $\langle \bar{s}, \bar{s}' \rangle \in \bar{\tau}(\bar{e})$, we get the simulation proof obligation rule **SIM** of [4, Sect. 5.2.6]. However, (50) is more general, in that $\bar{\tau}^*$ might be the identity $\bar{\tau}^0$, which is handled in [4, Sect. 14.6.1] on slitting an abstract event. $\bar{\tau}^*$ might also correspond to multiple abstract transitions $\bar{\tau}^n$, $n > 1$, as handled in **MRG** [4, Sect. 14.6.2] for merging several abstract events.

The transitions $\tau(e)$ and $\bar{\tau}(\bar{e})$ considered in (50) are defined by events in Event-B [4, Sect. 5.1.7]. The “Clause “with” in an event contains the *witness* of the corresponding abstract event. A witness has to be provided in a refining event *for each disappearing parameter* of the abstract event and *for each disappearing variable* assigned in a “non-deterministic way” in the abstract event. The non-deterministic witness proof obligation rule **WFIS** in [4, Sect. 5.2.10] “ensures that each witness proposed in the witness predicate of a concrete event indeed exists” [4, p. 201]. This corresponds to the condition $\exists \langle \bar{s}', \bar{e}' \rangle . \langle \langle s', e' \rangle, \langle \bar{s}', \bar{e}' \rangle \rangle \in \rho$ in (50).

Finally, (50) requires the event convergence (49) which is ensured by the variant proof obligation rule VAR [4, Sect. 5.2.9] which applies to both the concrete and abstract machines.

6 A General Induction Principle

The ideas of Jean-Raymond on abstraction that governed the design of Event-B are of general interest, both in refinement and verification. Let us consider

- $\underline{S}$ and $\overline{S}$ be sets of states;
- $\underline{\mathbb{T}}$ and $\overline{\mathbb{T}}$ be the finite or infinite traces over the respective states $\underline{S}$ and $\overline{S}$;
- $\underline{T} \in \wp(\underline{\mathbb{T}})$ and $\overline{T} \in \wp(\overline{\mathbb{T}})$ be trace semantics;
- $R_s \in \wp(\wp(\underline{\mathbb{T}}) \times \wp(\overline{\mathbb{T}}))$ be a relation between trace semantics;
- $\underline{P} \in \wp(\wp(\underline{\mathbb{T}}))$ and $\overline{P} \in \wp(\wp(\overline{\mathbb{T}}))$ be semantic hyperproperties;
- $R_p \in \wp(\wp(\wp(\underline{\mathbb{T}})) \times \wp(\wp(\overline{\mathbb{T}})))$ be a relation between semantic hyperproperties.

A correctness proof can be formalized by the following induction principle

$$\frac{\overline{T} \in \overline{P}}{\underline{T} \in \underline{P}}, \quad \langle \underline{T}, \overline{T} \rangle \in R_s, \quad \langle \underline{P}, \overline{P} \rangle \in R_p \quad (51)$$

Notice that the side conditions of (51) cannot be integrated in the premise (as often done in computer science, for example in Hoare logic where side conditions appear in the premise [15, Sect. 26.7]). If $R_s = \emptyset$ or $R_p = \emptyset$ then (51) is not applicable whereas if incorporated in the premise, (51) becomes the obviously wrong axiom $\frac{\emptyset}{\underline{T} \in \underline{P}}$.

Soundness of (51) states that R_s and R_p should be chosen so that the hyperproperty $\overline{P}$ of $\overline{T}$ should imply the hyperproperty $\underline{P}$ of $\underline{T}$. Conversely, completeness of the induction principle (51) states that R_s and R_p should be chosen so that if the hyperproperty $\underline{P}$ of $\underline{T}$ holds then this should be provable by showing that $\overline{T}$ has hyperproperty $\overline{P}$. Notice that R_p may incorporate the abstraction of hyperproperties to traces properties in $\wp(\overline{\mathbb{T}})$ and $\wp(\underline{\mathbb{T}})$.

An early example is Milner's introduction of the notion of simulation (and bisimulation called "mutual simulation") [32] with its "application to flowchart programs" in Sect. 4 to "demonstrate a simulation between two programs in a manner which bears a close relation to Floyd's method for proving correctness of a single program". The idea is further developed by Burstall [9]. Another example based on (51) is transformation in Continuation Passing Style (CPS) of higher-order functional programs before strictness analysis [28] which improves precision for the same abstract domain. In many cases however, (51) has one of R_s or R_p which is equality.

6.1 Verification by Property Abstraction. In verification by abstract interpretation one reasons on a given program with semantics $T = \underline{T} = \overline{T}$ so that $R_s = (=)$ is the identity in (51).

The condition for (51) to be sound when $R_s = (=)$ is that the premise of (51) should imply its conclusion when the side-condition $\langle \underline{P}, \overline{P} \rangle \in R_p$ holds, that is

$$\begin{aligned}
& \forall P, P' . (\langle P, P' \rangle \in R_p) \Rightarrow (\forall T . (T \in P') \Rightarrow T \in P) \\
\Leftrightarrow & R_p \subseteq \{ \langle P, P' \rangle \mid \forall T . T \in P' \Rightarrow T \in P \} && \text{\textit{\textup{[def. } $\subseteq$}}\textit{}} \\
\Leftrightarrow & R_p \subseteq \{ \langle P, P' \rangle \mid P' \subseteq P \} && \text{\textit{\textup{[def. } $\subseteq$}}\textit{}} \\
\Leftrightarrow & R_p \subseteq (\subseteq) && \text{\textit{\textup{[def. } $\subseteq$}}\textit{}}
\end{aligned}$$

Completeness of (51) when $R_s = (=)$ means that when the side condition $\langle \underline{P}, \overline{P} \rangle \in R_p$ holds (so that the rule is applicable), if the conclusion is true, this should be provable by (51), so that the premise should hold. So we should have

$$\begin{aligned}
& \forall P, P' . (\langle P, P' \rangle \in R_p) \Rightarrow (\forall T . (T \in P) \Rightarrow T \in P') \\
\Leftrightarrow & R_p \subseteq (\subseteq) && \text{\textit{\textup{[def. } $\subseteq$}}\textit{}}
\end{aligned}$$

The method is sound and complete if and only iff $R_p = (=)$ i.e. when no concrete information is lost in the abstract, which is the case only for isomorphic abstractions.

6.2 Verification by Program Refinement. As suggested by the explanation of the Event-B method in section 4, program verification can be done by refinement [24]. In verification by refinement, one reasons with respect to a fixed specification $P = \underline{P} = \overline{P}$ so that $R_p = (=)$ is the identity in (51);

The condition for (51) to be sound when $R_p = (=)$ is that the premise of (51) should imply its conclusion when the side-condition $\langle \underline{T}, \overline{T} \rangle \in R_s$ holds, that is, for all P and $\overline{P}$,

$$\begin{aligned}
& \forall T, \overline{T} . \langle T, \overline{T} \rangle \in R_s \Rightarrow (\overline{T} \in \overline{P} \Rightarrow T \in P) \\
\Leftrightarrow & \forall T . (\exists \overline{T} . \langle T, \overline{T} \rangle \in R_s \wedge \overline{T} \in \overline{P}) \Rightarrow T \in P && \text{\textit{\textup{[def. } $\Rightarrow$}}\textit{}} \\
\Leftrightarrow & \{ T \mid \exists \overline{T} . \langle T, \overline{T} \rangle \in R_s \wedge \overline{T} \in \overline{P} \} \subseteq P && \text{\textit{\textup{[def. } $\subseteq$}}\textit{}} \\
\Leftrightarrow & \text{pre}[R_s]\overline{P} \subseteq P && \text{\textit{\textup{[def. pre}}}\textit{}} \\
\Leftrightarrow & \overline{P} \subseteq \widetilde{\text{post}}[R_s]P && \text{\textit{\textup{[15, (12.23)]}}\textit{}}
\end{aligned}$$

The condition for (51) to be complete when $R_p = (=)$ is that if the conclusion of (51) holds, then this must be provable by the proof principle, so that the premise should hold, that is, for all P and $\overline{P}$,

$$\begin{aligned}
& \forall T, \overline{T} . (\langle T, \overline{T} \rangle \in R_s \wedge T \in P \Rightarrow \overline{T} \in \overline{P}) \\
\Leftrightarrow & \forall \overline{T} . (\exists T . \langle T, \overline{T} \rangle \in R_s \wedge T \in P) \Rightarrow \overline{T} \in \overline{P} && \text{\textit{\textup{[def. } $\Rightarrow$}}\textit{}} \\
\Leftrightarrow & \{ \overline{T} \mid \exists T . T \in P \wedge \langle T, \overline{T} \rangle \in R_s \} \subseteq \overline{P} && \text{\textit{\textup{[def. } $\subseteq$ and commutativity}}\textit{}} \\
\Leftrightarrow & \text{post}[R_s]P \subseteq \overline{P} && \text{\textit{\textup{[def. post}}}\textit{}} \\
\Leftrightarrow & P \subseteq \widetilde{\text{pre}}[R_s]\overline{P} && \text{\textit{\textup{[15, (12.22)]}}\textit{}}
\end{aligned}$$

It follows that (51) is sound and complete when $R_p = (=)$ if and only if $\text{pre}[R_s]\overline{P} \subseteq P \subseteq \widetilde{\text{pre}}[R_s]\overline{P}$ (or equivalently $\text{post}[R_s]P \subseteq \overline{P} \subseteq \widetilde{\text{post}}[R_s]P$).

6.3 Program Refinement. In machine/program refinement, the abstract specification $\overline{P}$ is the abstract machine/program semantics $\overline{T}$ itself so $\overline{P} = \{\overline{T}\}$. The refined machine/program semantics $\underline{T}$ must be a subset of the abstract machine/program semantics, so $R_s = \{\langle \underline{T}, \overline{T} \rangle \mid \emptyset \neq \underline{T} \subseteq \overline{T}\}$ and $R_p = \{\langle \underline{P}, \overline{T} \rangle \mid \underline{P} \in \wp(\overline{T}) \setminus \{\emptyset\}\}$ in (51). In this case, (51) becomes “design the refined machine/program semantics $\underline{T}$ such that $\emptyset \neq \underline{T} \subseteq \overline{T}$ ” which is the simple trace refinement in section 4. Abstraction or concretization functions can be used to include the other refinements defined in this section.

6.4 Static Analysis. As noted in (38), the rôles are inverted in static analysis where the concrete program semantics $\overline{T}$ is the specification, $\overline{P} = \{\overline{T}\}$ is the collecting semantics, and the abstract semantics $\underline{T}$ should be an over-approximation $\underline{T} \supseteq \overline{T}$, up to either a Galois connection, a concretization, or an abstraction function [21], which can be formalized by appropriate choices of R_s and R_p in (51).

7 Conclusion

We have sketched the semantics of Events-B in terms of abstract interpretation and defined the semantics of machines and the semantics of refinement as the concretization of a Galois connection. Event-B concentrates on trace properties. Following [26,27,16], the proof rules of [4, Ch. 9] can be derived by abstract interpretation. The abstract interpretation approach [22] might also be useful to extend Event-B to consider hyperproperties [11].

Moreover, in abstract interpretation knowing the concrete semantics and the abstraction yields the abstract semantics by calculational design (e.g. [12,22]). This means that making abstractions explicit, as we did, could yield refinements by calculation design (preferred to manual refinement followed by verification).

Finally, we have considered the abstraction/refinement of both semantics and properties and shown in this context that verification and refinement are dual. By this duality, the study of only one of verification or refinement is necessary since the results immediately apply, by duality, to the other. However, the communities, techniques, and literatures are, in part, but mostly disconnected.

Acknowledgements. I thank Dominique Méry for pointing out my misconceptions of Event-B in a first version of this paper⁴.

⁴ Among others, I misinterpreted the parameters in the **any** clause of events [3, Fig. 5.10] as input or output parameters to communicate with the environment, whereas the system is closed in Event-B and these parameters are existentially quantified. Curiously, the paper is still coherent with this misconception and enables Event-B to design open systems, provided event convergence is defined not to depend on these parameters.

References

1. Abrial, J.: Data semantics. In: Klimbie, J.W., Koffeman, K.L. (eds.) IFIP Working Conference Data Base Management. pp. 1–60. North-Holland (1974)
2. Abrial, J.: The B-book - assigning programs to meanings. Cambridge University Press (1996)
3. Abrial, J.: Extending B without changing it (for developing distributed systems). In: Habrias, H. (ed.) Proceedings of the 1st Conference on the B method. pp. 169–191. Springer (Nov 1996)
4. Abrial, J.: Modeling in Event-B: System and Software Engineering. Cambridge University Press (2010)
5. Abrial, J., Butler, M.J., Hallerstede, S., Hoang, T.S., Mehta, F., Voisin, L.: Rodin: an open toolset for modelling and reasoning in Event-B. *Int. J. Softw. Tools Technol. Transf.* **12**(6), 447–466 (Nov 2010). <https://doi.org/10.5555/1895404.1895406>
6. Abrial, J., Cansell, D.: Click’n prove: Interactive proofs within set theory. In: TPHOLs. *Lecture Notes in Computer Science*, vol. 2758, pp. 1–24. Springer (2003)
7. Abrial, J., Mussat, L.: Introducing dynamic constraints in B. In: *B. Lecture Notes in Computer Science*, vol. 1393, pp. 83–128. Springer (1998)
8. Alglave, J., Cousot, P.: Syntax and analytic semantics of LISA. *CoRR abs/1608.06583* (Aug 2016), <http://arxiv.org/abs/1608.06583>
9. Burstall, R.M.: An algebraic description of programs with assertions, verification and simulation. In: *Proving Assertions About Programs*. pp. 7–14. ACM (1972). <https://doi.org/10.1145/942578.807068>
10. Chandy, K.M., Misra, J.: *Parallel Program Design: A Foundation*. Addison-Wesley (Jan 1988)
11. Clarkson, M.R., Schneider, F.B.: Hyperproperties. *J. Comput. Secur.* **18**(6), 1157–1210 (2010). <https://doi.org/10.3233/JCS-2009-0393>
12. Cousot, P.: The calculational design of a generic abstract interpreter. In: Broy, M., Steinbrüggen, R. (eds.) *Calculational System Design*. NATO ASI Series F. IOS Press, Amsterdam (1999)
13. Cousot, P.: A formal introduction to abstract interpretation. In: Pretschner, A., Müller, P., Stöckle, P. (eds.) *Calculational System Design*. NATO SPS, Series D, Vol. 53. IOS Press, Amsterdam (1999)
14. Cousot, P.: Constructive design of a hierarchy of semantics of a transition system by abstract interpretation. *Theor. Comput. Sci.* **277**(1–2), 47–103 (2002). [https://doi.org/10.1016/S0304-3975\(00\)00313-3](https://doi.org/10.1016/S0304-3975(00)00313-3)
15. Cousot, P.: *Principles of Abstract Interpretation*. MIT Press, 1 edn. (2021)
16. Cousot, P.: Calculational design of [in]correctness transformational program logics by abstract interpretation. *Proc. ACM Program. Lang.* **8**(POPL), 175–208 (2024). <https://doi.org/10.1145/3632849>
17. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: *POPL*. pp. 238–252. ACM (1977). <https://doi.org/10.1145/512950.512973>
18. Cousot, P., Cousot, R.: Systematic design of program analysis frameworks. In: *POPL*. pp. 269–282. ACM Press (1979). <https://doi.org/https://dl.acm.org/doi/10.1145/3728920>
19. Cousot, P., Cousot, R.: Induction principles for proving invariance properties of programs. In: Néel, D. (ed.) *Tools & Notions for Program Construction: an Advanced Course*. pp. 75–119. Cambridge University Press, Cambridge, UK (Aug 1982)

20. Cousot, P., Cousot, R.: 'a la Floyd' induction principles for proving inevitability properties of programs. In: Nivat, M., Reynolds, J. (eds.) *Algebraic methods in semantics*. pp. 277–312. Cambridge University Press, Cambridge, UK (Dec 1985)
21. Cousot, P., Cousot, R.: Abstract interpretation frameworks. *J. Log. Comput.* **2**(4), 511–547 (1992). <https://doi.org/10.1093/LOGCOM/2.4.511>
22. Cousot, P., Wang, J.: Calculational design of hyperlogics by abstract interpretation. *Proc. ACM Program. Lang.* **9**(POPL), 446–478 (2025). <https://doi.org/10.48550/arXiv.2411.11113>
23. Cousot, R.: *Fondements des méthodes de preuve d'invariance et de fatalité de programmes parallèles* (in French). Thèse d'État ès sciences mathématiques, Nancy, Institut national polytechnique de Lorraine (15 November 1985), <https://pcousot.github.io/publications/RadhiaCousotTheseEsSciences.PDF>
24. Dijkstra, E.W.: A constructive approach to the problem of program correctness. *BIT Numerical Mathematics* **8**(3), 174–186 (Sep 1968). <https://doi.org/10.1007/BF01933419>
25. Dijkstra, E.W.: On weak and strong termination. In: *Selected Writings on Computing: A Personal Perspective*. pp. 355–357. Texts and Monographs in Computer Science, Springer (1982), originally published as EWD673 (1978)
26. D'Silva, V.V., Haller, L., Kroening, D.: Satisfiability solvers are static analysers. In: *SAS. Lecture Notes in Computer Science*, vol. 7460, pp. 317–333. Springer (2012). https://doi.org/10.1007/978-3-642-33125-1_22
27. D'Silva, V.V., Haller, L., Kroening, D.: Abstract satisfaction. In: *POPL*. pp. 139–150. ACM (2014). <https://doi.org/10.1145/2535838.2535868>
28. Filho, J.M., Burn, G.L.: Continuation passing transformation and abstract interpretation. In: Burn, G.L., Gay, S.J., Ryan, M. (eds.) *Theory and Formal Methods 1993, Proceedings of the First Imperial College Department of Computing Workshop on Theory and Formal Methods*, Isle of Thorns Conference Centre, Chelwood Gate, Sussex, UK, 29–31 March 1993. pp. 247–259. *Workshops in Computing*, Springer (1993)
29. Hoang, T.S.: An introduction to the Event-B modelling method. In: *Industrial Deployment of System Engineering Methods*, pp. 211–236. Springer (2013)
30. Lamport, L.: *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Professional (Jul 2002)
31. Méry, D., Merz, S.: Analysis of a termination detection algorithm using Event-B and TLA+. This volume (2026)
32. Milner, R.: An algebraic definition of simulation between programs. In: *IJCAI*. pp. 481–489. William Kaufmann (1971). <https://doi.org/10.5555/1622876.1622926>
33. Méry, D., Singh, N.K.: Event B, chap. 10, pp. 253–298. *Formal Methods Applied to Complex Systems: Implementation of the B Method*, J.-L. Boulanger (Ed.), Wiley, 1st edn. (2014)
34. Tarski, A.: A lattice theoretical fixpoint theorem and its applications. *Pacific J. of Math.* **5**, 285–310 (1955). <https://doi.org/10.2140/pjm.1955.5.285>
35. Voisin, L., Abrial, J.: The Rodin platform has turned ten. In: *ABZ. Lecture Notes in Computer Science*, vol. 8477, pp. 1–8. Springer (2014)

If $\dots \pi_{i+1} \pi'_{i+2} \dots, \pi'_{i+k} \dots$ is the longest possible sequence of events e on any trace going through the state π_{i+1} , so that the event of π'_{i+k+1} is not e , then,

$$V(\pi_{i+1}) = \alpha_v \langle S, E \rangle(\pi_{i+1}) = k - (i + 1).$$

Let us now consider the state π_i . There are two cases.

- Either there is a trace $\dots \pi_i \pi''_{i+2} \dots, \pi''_{i+\ell} \dots$ with $\ell > k$ consecutive events $\mathbf{e}$, in which case $V(\pi_i) = \alpha_v \langle S, E \rangle (\pi_i) = \ell - i > k - (i + 1)$ so that (30) holds;
- Or else all such traces have less than k consecutive events $\mathbf{e}$, in which case, by history independence $\text{Ffus}(\mathbf{T}) = \mathbf{T}$, $\dots \pi_i \pi_{i+1} \pi'_{i+2} \dots, \pi'_{i+k} \dots$ is also a trace through π_i , and therefore $V(\pi_i) = \alpha_v \langle S, E \rangle (\pi_i) = k - i > k - (i + 1)$ so that (30) holds again. $\square$

Proof (of (25)). Because of event convergence (13), $\alpha_v\langle S, E \rangle(T)$ is always well-defined since the number of successive computation steps for a given event e on any trace of T is bounded ⁵.

$$\begin{aligned}
& \alpha_v \langle S, E \rangle (T) \preceq V \\
& \Leftrightarrow \forall \langle s, e \rangle . \alpha_v \langle S, E \rangle (T) \langle s, e \rangle \leq V \langle s, e \rangle \quad \wr \text{pointwise def. } \preceq \\
& \Leftrightarrow \forall \langle s, e \rangle . \max \{ j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T . \forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle \} \leq V \langle s, e \rangle \quad \wr \text{def. (28) of } \alpha_v \\
& \Leftrightarrow \forall \langle s, e \rangle . \forall n \in \{ j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T . \forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle \} . n \leq V \langle s, e \rangle \quad \wr \text{def. max} \\
& \Leftrightarrow \forall \langle s, e \rangle . \forall j > i \in \mathbb{N} . (\exists \pi \in T . \forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle) \Rightarrow (j - i \leq V \langle s, e \rangle) \quad \wr \text{def. } \leq \\
& \Leftrightarrow \forall \langle s, e \rangle . \forall j > i \in \mathbb{N} . \forall \pi \in T . (\forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle) \Rightarrow (j - i \leq V \langle s, e \rangle) \quad \wr \text{def. } \Rightarrow \\
& \Leftrightarrow \forall \pi \in T . \forall \langle s, e \rangle . \forall j > i \in \mathbb{N} . (\forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle) \Rightarrow (j - i \leq V \langle s, e \rangle) \quad \wr \text{def. } \forall \\
& \Leftrightarrow T \subseteq \{ \pi \mid \forall \langle s, e \rangle . \forall j > i \in \mathbb{N} . (\forall k \in [i, j] . \exists s_k . s_i = s \wedge \pi_k = \langle s_k, e \rangle) \Rightarrow (j - i \leq V \langle s, e \rangle) \} \quad \wr \text{def. } \subseteq \\
& \Leftrightarrow T \subseteq \{ \pi \mid \text{cvgt} \langle S, E \rangle (\pi, V) \} \quad \wr \forall s . \forall i . \exists s_i . s_i = s \text{ always holds} \\
& \Leftrightarrow T \subseteq \gamma_v \langle S, E \rangle (V) \quad \wr \text{def. (29) of } \gamma_v \quad \square
\end{aligned}$$

Proof (of (34)). The Cartesian product of Galois connections (24) and (25) is a Galois connection (34). For the isomorphism, we have

$$\begin{aligned}
& - \alpha_t \dot{\times} \alpha_v \langle S, E \rangle \circ \gamma_t \dot{\times} \gamma_w \langle S, E \rangle (\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle) \\
= & \text{let } T = \{ \pi \in \mathbb{T}_{\omega} \langle S, E \rangle \mid \pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \wedge \forall \langle s, e \rangle = \pi_i, \langle s', e' \rangle = \pi_{i+1}. \langle s, s' \rangle \in \tau(e) \wedge \text{cvg}_t \langle S, E \rangle (\pi, V) \} \text{ in} \\
& \quad \{ \langle s \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e \in E. \pi_i = \langle s, e \rangle \rangle, \\
& \quad \{ \langle s \mid \exists \pi \in T. \exists e \in E. \pi_0 = \langle s, e \rangle \rangle, \\
& \quad \{ \langle e \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists s \in S. \pi_i = \langle s, e \rangle \rangle, \\
& \quad \lambda e \cdot \{ \langle s, s' \rangle \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e' \in E. \pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle \} \}, \\
& \quad \lambda \langle s, e \rangle \cdot \max \{ j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T. \forall k \in [i, j]. \exists s_k. s_i = s \wedge \pi_k = \langle s_k, e \rangle \} \} \\
& \qquad \wr ((33), (32), \text{and def. function composition } \circ_s) \\
= & \langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle
\end{aligned}$$

⁵ Observe that the alternative definition of (13) in remark 1, which is unbounded, would require transfinite ordinals.

$\lambda(\dot{=})$ follows from the Galois connection (34). By antisymmetry, it remains to prove the converse.

$(\dot{=})$ is proved componentwise, by showing that

- If $\pi \in T$ and $i \in \mathbb{N}$ are such that $\pi_i = \langle s, e \rangle$ then by def. of T and (22) all states and events are in the invariant $\text{inv} \times \text{evts}$ proving that $\{s \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e \in E. \pi_i = \langle s, e \rangle\} \subseteq \text{inv}$ and $\{e \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists s \in S. \pi_i = \langle s, e \rangle\} \subseteq \text{evts}$.
- If $\pi \in T$ then by definition of T , $\pi_0 \in \text{init} \times \text{evts}$ so if $\exists e \in E. \pi_0 = \langle s, e \rangle$ then $s \in \text{init}$ proving that $\{s \mid \exists \pi \in T. \exists e \in E. \pi_0 = \langle s, e \rangle\} \subseteq \text{init}$
- If $\pi \in T$, $\pi_i = \langle s, e \rangle$, and $\pi_{i+1} = \langle s', e' \rangle$ then $\langle s, s' \rangle \in \tau(e)$, proving that $\lambda e \cdot \{\langle s, s' \rangle \mid \exists \pi \in T. \exists i \in \mathbb{N}. \exists e' \in E. \pi_i = \langle s, e \rangle \wedge \pi_{i+1} = \langle s', e' \rangle\} \subseteq \tau$
- Given any $\pi \in T$, $\langle s, e \rangle \in \langle \text{inv}, \text{evts} \rangle$, and $i \in \mathbb{N}$ such that $\pi_i = \langle s, e \rangle$, the definition of T implies that $\forall j > i. \forall k \in [i, j]. \exists s_k. s_i = s \wedge \pi_k = \langle s_k, e \rangle$.
 $\max\{j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T. \forall k \in [i, j]. \exists s_k. s_i = s \wedge \pi_k = \langle s_k, e \rangle\} \leq V \langle s, e \rangle$.
Q.E.D.

- By the Galois connection (34), we know that $\forall T \in \mathbb{T}^{\text{EvB}} \langle S, E \rangle. T \subseteq \gamma_t \dot{\times} \gamma_v \langle S, E \rangle \circ \alpha_t \dot{\times} \alpha_v \langle S, E \rangle (T)$ where $T \in \mathbb{T}^{\text{EvB}} \langle S, E \rangle$.

To prove the converse, assume that $\pi \in \gamma_t \dot{\times} \gamma_v \langle S, E \rangle \circ \alpha_t \dot{\times} \alpha_v \langle S, E \rangle (T)$ where $\gamma_t \dot{\times} \gamma_v \langle S, E \rangle \circ \alpha_t \dot{\times} \alpha_v \langle S, E \rangle (T)$ is given by (32) and (33). We must prove that $\pi \in T$ where $T \in \mathbb{T}^{\text{EvB}} \langle S, E \rangle$.

By (33), this means that we can assume that $\pi_0 \in \text{init} \times \text{evts} \wedge \forall i \in \mathbb{N}. \pi_i \in \text{inv} \times \text{evts} \wedge \forall \langle s, e \rangle = \pi_i, \langle s', e' \rangle = \pi_{i+1}. \langle s, s' \rangle \in \tau(e) \wedge \text{cvgt} \langle S, E \rangle (\pi, V)$ where $\langle \text{inv}, \text{evts}, \text{init}, \tau, V \rangle$ is defined by (32), that is (17). Assume, by reductio ad absurdum, that $\pi \notin T$. Then there is an earliest trace transition $\langle s, e \rangle$ such that $\pi = \pi_1 \langle s, e \rangle \langle s', e' \rangle \pi_2 \notin T$ with some $\pi_1 \langle s, e \rangle \langle s'', e'' \rangle \pi_2' \in T$.

We cannot have π_1 empty since then $\langle s, e \rangle \langle s'', e'' \rangle \pi_2' \in T$ implies that s is an initial state so that, by definition of the initial states in (32), this does not prevent $\pi = \pi_1 \langle s, e \rangle \langle s', e' \rangle \pi_2 = \langle s, e \rangle \langle s', e' \rangle \pi_2$ to belong to T .

It follows that π_1 must be not empty. In that case $\langle s, e \rangle \langle s', e' \rangle$ is a valid transition while $\langle s, e \rangle \langle s', e' \rangle$ is not in (32), meaning either that $\forall \pi \in T. \forall i \in \mathbb{N}. \forall e'' \in E. \pi_i = \langle s, e \rangle \Rightarrow \pi_{i+1} \neq \langle s', e'' \rangle$, a contradiction or else $\neg \text{cvgt} \langle S, E \rangle (\pi, \lambda \langle s, e \rangle \cdot \max\{j - i \mid j > i \in \mathbb{N} \wedge \exists \pi \in T. \forall k \in [i, j]. \exists s_k. s_i = s \wedge \pi_k = \langle s_k, e \rangle\})$, another contradiction. By contradiction, we conclude that $\pi \in T$. $\square$